

HW 3 Rubric

110/100 points to get + extra credit.

1. Adherence to Shared Interface (19 pts)

- [7 pts] **OSS-APIs Implementation:** The team has refactored their original component to strictly implement the standardized **ABC** (Abstract Base Class) defined by their vertical (Chat, Ticket, or AI).
 - [7 pts] **External Interface Usage:** The team imports and uses the official **ABCs** for the *other* two verticals they are integrating, rather than defining their own ad-hoc interfaces.
 - [5 pts] **No Leaky Abstractions:** The implementation details of specific providers (e.g., Jira specific fields, Slack specific events) do not leak into the shared interface layer.
-

2. System Integration & Architecture (13 pts)

- [5 pts] **Dependency Injection (IoC):** The application uses a clear DI pattern to inject concrete implementations into the core logic.
 - *Check:* Can the system (theoretically) swap between providers by changing configuration/factories without rewriting business logic?
 - [5 pts] **Provider Support:** The submission demonstrates successful integration with **at least one** external provider for each of the **two** required verticals.
 - [3 pts] **The "User Flow" Loop:** The full pipeline functions correctly:
 1. User Input (Chat)
 2. Routing/Reasoning (AI)
 3. Execution (Ticket Service)
 4. Response (Chat)
-

3. Application Logic (5 pts)

- [3 pts] **Tool Calling / Structured Output:** The AI extracts specific data (e.g., `title="Fix bug"`) and returns a structured tool call or JSON object, rather than just free text.

- [2 pts] **Error Handling:** The application handles failures gracefully (e.g., if the Ticket API is down, the Chat user receives a friendly error message, not a stack trace).
-

4. Infrastructure & Deployment (IaC) (13 pts)

Note: We will not run this part locally; grading is based on the evidence shown in your video submission and a pass over your code.

- [5 pts] **Infrastructure as Code:** Resources (servers, containers, databases, env vars) are provisioned with TF or similar.
 - [4 pts] **Deployment Capability:** The application *CAN BE* deployed to a remote environment (not just running on `localhost`). The IaC scripts work.
 - [4 pts] **Configuration Management:** Secrets (API keys, tokens) are managed securely via environment variables or a secrets manager, not hardcoded.
-

5. Observability & Telemetry (10 pts)

Note: We will not run this part locally; grading is based on the evidence shown in your video submission and a pass over your code.

- [4 pts] **Instrumentation:** The application emits telemetry data for key operations.
 - [3 pts] **Required Metrics:** The dashboard or logs clearly show **at least 2** of the following:
 - Request Latency
 - Success Rate
 - Failure Rate
 - [3 pts] **Visualization:** Evidence (screenshot/video) of a monitoring platform (Datadog, Prometheus/Grafana, CloudWatch, etc.) visualizing this data.
-

6. Testing Strategy (25 pts)

- [8 pts] **Integration Tests:** Tests exist that verify the interaction between two specific components.
- [8 pts] **End-to-End (E2E) Tests:** Tests run the full application entry point (subprocess or deployed env) and verify the complete user flow (Input Action Output) (this should use real creds set up as env vars secrets in circle ci)

- [5 pts] **Mocking/Fakes:** *Integration tests* do not hit live, expensive external APIs (OpenAI/Jira) unless specifically designated as a "live" integration test.
 - [4 pts] **CI Pipeline:** The CircleCI (or equivalent) config has been updated to run these new integration tests.
-

7. Code Quality & Repository Hygiene (10 pts)

Note: There are levels to this. If you have few errors you could still get 2 or 2.5 in the analyzers.

- [3.5 pts] **Strict Typing:** mypy (strict mode) passes across the integrated codebase.
 - [3.5 pt] **Linting:** ruff passes with no errors.
 - [3 pts] **Clean Commit History:** Commits follow the "feat/fix" convention and tell a story of the integration process.
-

8. Documentation & Deliverables (15 pts)

- [10 pts] **Video Demonstration:**
 - Explains the architecture.
 - Live demo of the functionality.
 - Walks through the CI pipeline.
 - Shows the Telemetry dashboard.
 - * 5 extra credit if you show a switch between to implementations.
- [2 pts] **Updated README:** Explains how to run the full integrated stack, how to set up the necessary credentials for all services, and how to deploy the IaC.
- [3 pts] **PR Quality:** The final Pull Request is clean, well-documented, and represents the final state of the project. Should not be to root (unless you started a new repo).